1. Design Philosophy

1.1 Overview

- Follow the **Keep It Simple** and **You Ain't Gonna Need It** philosophies.
- Code should be designed to be efficient and easy to maintain and support, while also balancing the objectives of **DRY** (don't repeat yourself) coding philosophy.
- Code must be easy to follow and debug for future developers and support staff. Keep in mind the next person to look at your code may be someone with less experience than yourself.
- Do not over-architect solutions unnecessarily.
- Do not add needless layers of abstraction or try to throw in every design pattern in the big blue book of design patterns.
- Do add abstraction and use appropriate design patterns when necessary to keep the code DRY, but keep in mind simplicity and supportability. The future support team should not have to jump through 50 classes to understand what the code is doing. Yes, this is a bit of a balancing act.
- Code should be written from a ".NET first" approach, prioritizing .NET code over SQL. This is because it is more efficient for RA to be able to scale out RA code to the SMs and IIS as need, whereas SQL can become a database bottleneck. Try to avoid writing logic in SQL as much as possible. Avoid massive 3000 line SQL SPs that can't multi-thread and are impossible to follow by anyone except the original author.

1.2 Classes

- Only have one class per file
- Using statements should go outside the namespace
    - Example:

```
using System;
using System.Data;

namespace ZZZ.Server.Accounting.Reports
{

}
```

1.3 Methods

- Use **PascalCase** for function, property, event, and class names
- All methods should be intuitively named based on their functionalities
- Custom services should also conform to the same naming convention
- Example: `ReadDataRowsFromExcelFile`
- Remember to keep it DRY (don't repeat yourself)
    - If you are writing basically the same code more than once in a class, can it be refactored into a method?
    - Is what you are doing already in a utility class, if not should it be? I.e. calling the registry
    - Is what you are doing a good candidate for an extension method?

1.4 Variables

- Variables should use **camelCase** capitalization style
- Variables and Enums should not start with an underscore, or another prefix such as m_
- Variable names should logically describe that the data stored in the variable is
    - Remember, other developers and support staff will look at this code and need to understand it quickly
    - Example: use **countOfFailedRecords** instead of a simple name such as **x** or **numRec**.
        - Using **x** or **i** is acceptable in a for loop
- Reference the guidelines published by Microsoft
    - C# Coding Conventions: http://msdn.microsoft.com/en-us/library/ff926074.aspx
    - Internal Coding Guidelines: https://blogs.msdn.microsoft.com/brada/2005/01/26/internal-coding-guidelines/
    - C# Coding Style https://github.com/dotnet/corefx/blob/master/Documentation/coding-guidelines/coding-style.md

- Note: these guides sometime conflict with each other

1.5 Database Access and Custom Entities

- Dapper should be used for custom entities connecting to the database.
- Do not use the old .NET datasets.
- Try to put all entity related calls to the database in the repository and service client.
- SQLUtility (custom or core) can be used when calling the DB for simple things
  - Note: there are DataType specific methods in the custom SQLUtility, such as:

    `ExecuteSqlGetSingleValueDatetime` try to use this instead of retrieving everything as a string, then having to convert it again in the class where you are calling SQLUtility
- Assume that the future support team will not have SQL access for updates/delete/inserts without going through change control. Any custom tables that need editing should have an editable report so they can be maintained through the GUI.
- Be very mindful of loops and anything that may result in multiple calls to the database. We need to avoid database contention.

  A common situation this appears is when you are looping through rows and need to get a value from the database. Consider retrieving all values you may need before the loop starts. For example, if you are creating an outbound invoice feed that has 500 rows and need to get an `InvoiceDisplayName` attribute for the Business Associate don't call the database 500 times to get `InvoiceDisplayName`. Instead call it once to get all the attribute values you are going to need.
  - Remember that the repository/service client methods will wind up calling the database as well, so build your repository methods in a way that will be efficient.